

DATA VORTEX 2026

ROUND 1 - PHASE 2

FINAL SQL QUERIES

One selected question at each required difficulty

Prepared from the project SQLite database using read-only SELECT analysis.

EASY QUESTION - E2 - Most Engaged Posts

Official question: Find the top 10 posts based on total engagement. Total engagement is defined as likes + shares + comments. Ignore posts where likes are missing.

```
SELECT
  p.post_id,
  p.user_id,
  p.platform,
  p.likes,
  p.shares,
  p.comments,
  p.likes + p.shares + p.comments AS total_engagement
FROM posts AS p
WHERE p.likes IS NOT NULL
ORDER BY total_engagement DESC, p.post_id ASC
LIMIT 10;
```

Executed successfully against **data/data_vortex.db**. Returned 10 row(s).

MEDIUM QUESTION - M1 - Which Locations Generate the Most Engagement?

Official question: Using both datasets, calculate the total engagement generated by users from each location. Return the location, number of posts, and total engagement. Rank locations from highest to lowest engagement.

```
SELECT
    u.location,
    COUNT(p.post_id) AS post_count,
    SUM(p.likes + p.shares + p.comments) AS total_engagement
FROM users AS u
JOIN posts AS p
    ON p.user_id = u.user_id
WHERE p.likes IS NOT NULL
GROUP BY u.location
ORDER BY total_engagement DESC, u.location ASC;
```

Executed successfully against **data/data_vortex.db**. Returned 33 row(s).

HARD QUESTION - H2 - Rank Users Within Their Location

Official question: For every location, rank users based on their total engagement. Return only the top 3 users from each location.

```
WITH user_engagement AS (  
  SELECT  
    u.user_id,  
    u.location,  
    u.follower_count,  
    COUNT(p.post_id) AS post_count,  
    SUM(p.likes + p.shares + p.comments) AS total_engagement  
  FROM users AS u  
  JOIN posts AS p  
    ON p.user_id = u.user_id  
 WHERE p.likes IS NOT NULL  
 GROUP BY u.user_id, u.location, u.follower_count  
) , ranked_users AS (  
  SELECT  
    user_id,  
    location,  
    follower_count,  
    post_count,  
    total_engagement,  
    DENSE_RANK() OVER (  
      PARTITION BY location  
      ORDER BY total_engagement DESC  
    ) AS engagement_rank  
  FROM user_engagement  
)  
SELECT  
  location,  
  engagement_rank,  
  user_id,  
  follower_count,  
  post_count,  
  total_engagement  
FROM ranked_users  
WHERE engagement_rank <= 3  
ORDER BY location ASC, engagement_rank ASC, user_id ASC;
```

Executed successfully against **data/data_vortex.db**. Returned 99 row(s).